Object-Oriented and Classical Software Engineering

8th Edition, WCB/McGraw-Hill, 2010

Stephen R. Schach
srs@vuse.vanderbilt.edu

CHAPTER 2

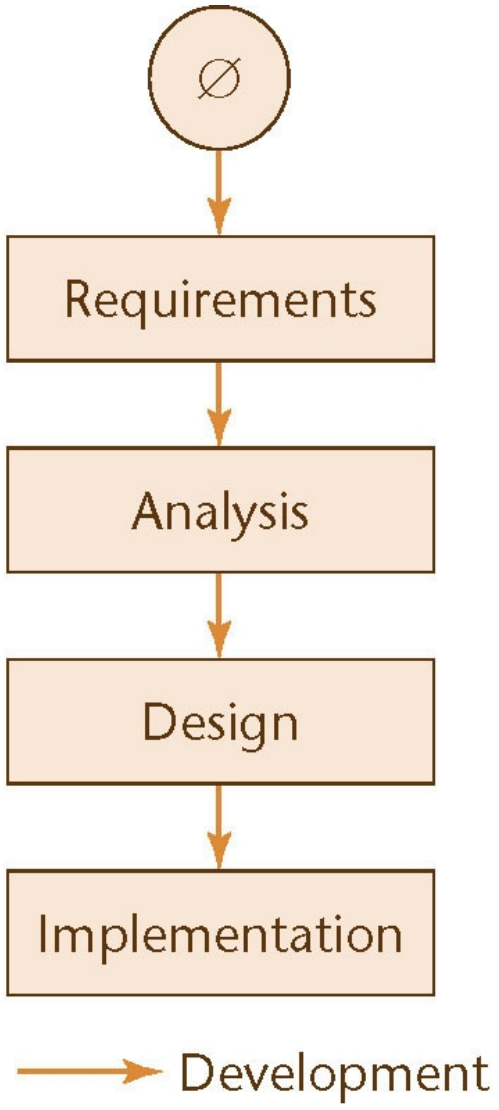
SOFTWARE LIFE-CYCLE MODELS

Overview

- Software development in theory
- Winburg mini case study
- Lessons of the Winburg mini case study
- Teal tractors mini case study
- Iteration and incrementation
- Winburg mini case study revisited
- Risks and other aspects of iteration and incrementation
- Managing iteration and incrementation
- Other life-cycle models
- Comparison of life-cycle models

Software Development in Theory

- Ideally, software is developed as described in Chapter 1
  - Linear
  - Starting from scratch

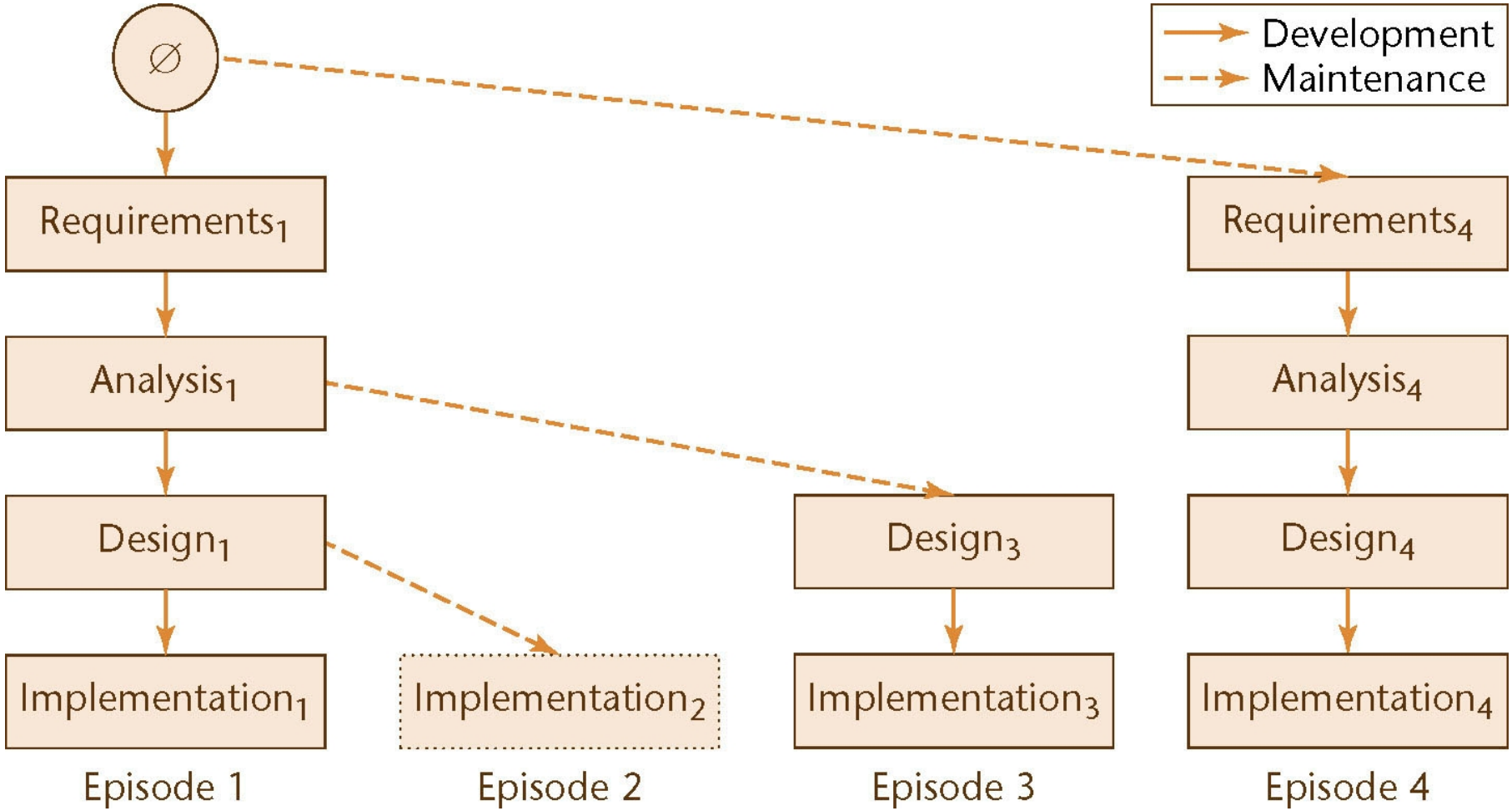


Software Development in Practice

- In the real world, software development is totally different
  - We make mistakes
  - The client's requirements change while the software product is being developed

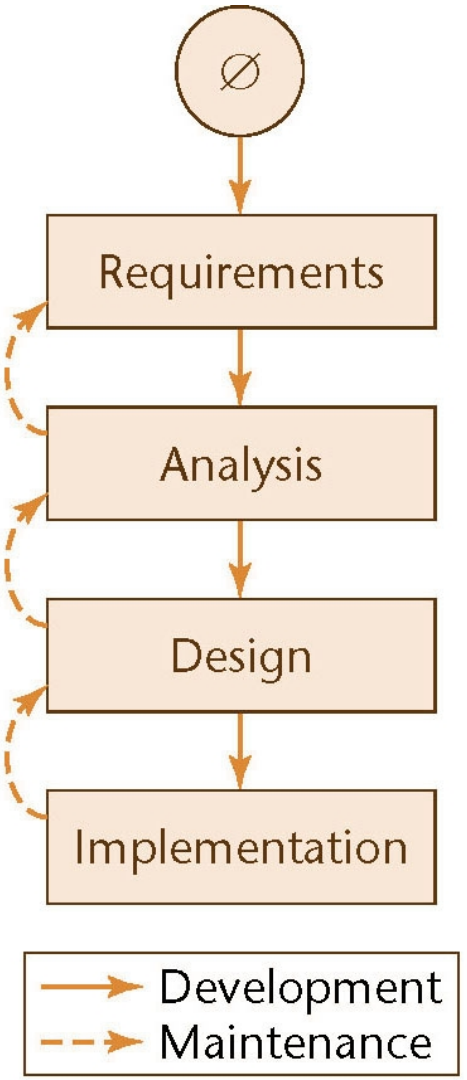
Evolution-Tree Model

- Winburg Mini Case Study



Waterfall Model

- The linear life cycle model with feedback loops
  - The waterfall model cannot show the order of events



Return to the Evolution-Tree Model

- The explicit order of events is shown
- At the end of each episode
  - We have a *baseline*, a complete set of *artifacts* (constituent components)
- Example:
  - Baseline at the end of Episode 3:
    - Requirements_1, Analysis_1, Design_3, Implementation_3

Moving Target Problem

- A change in the requirements while the software product is being developed
- Even if the reasons for the change are good, the software product can be adversely impacted
  - Dependencies will be induced

Moving Target Problem (contd)

- Any change made to a software product can potentially cause a *regression fault*
  - A fault in an apparently unrelated part of the software
- If there are too many changes
  - The entire product may have to be redesigned and reimplemented

Moving Target Problem (contd)

- Change is inevitable
  - Growing companies are always going to change
  - If the individual calling for changes has sufficient clout, nothing can be done about it
- There is no solution to the moving target problem

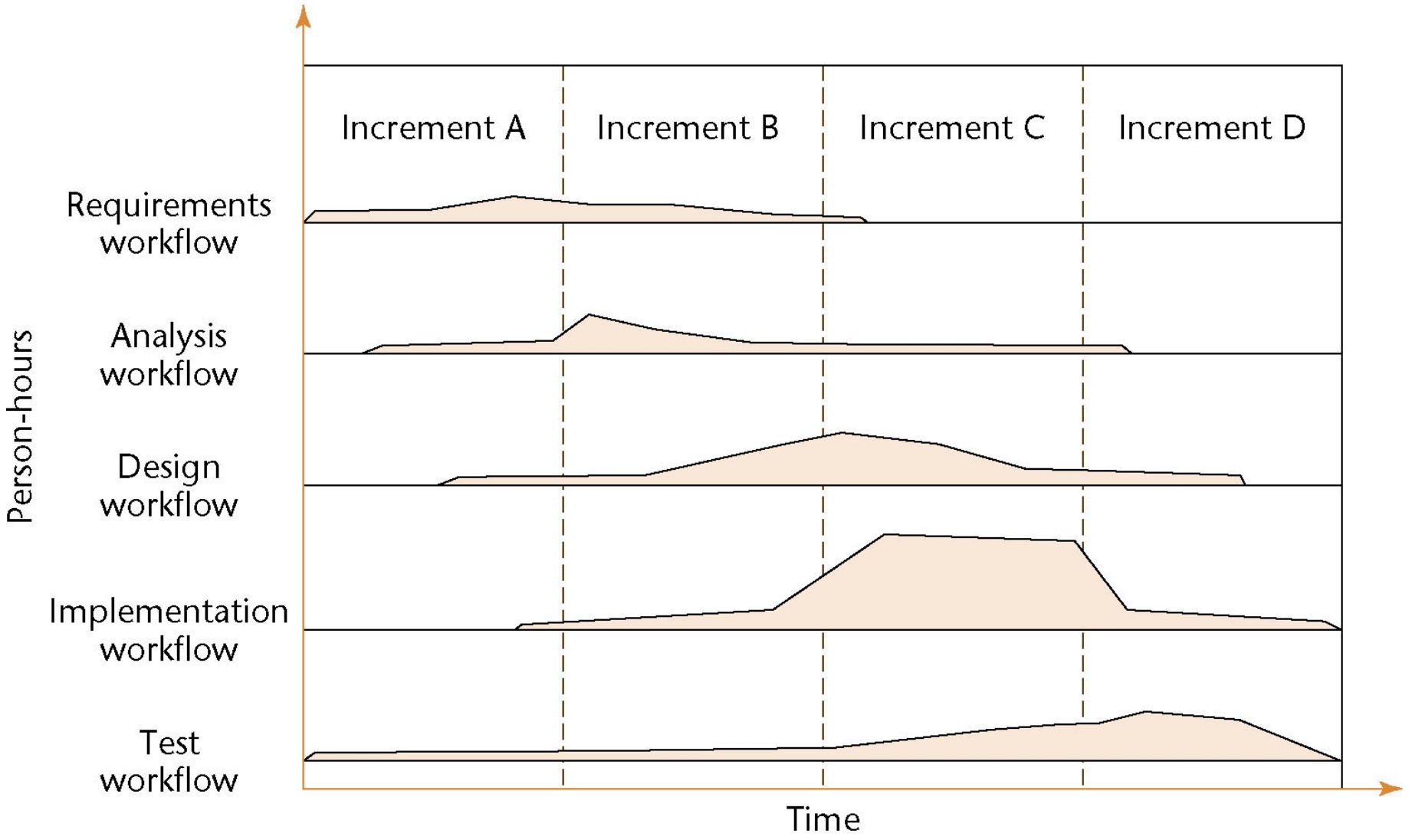
Iteration and Incrementation

- In real life, we cannot speak about “the analysis phase”
  - Instead, the operations of the analysis phase are spread out over the life cycle
- The basic software development process is iterative
  - Each successive version is intended to be closer to its target than its predecessor

Miller's Law

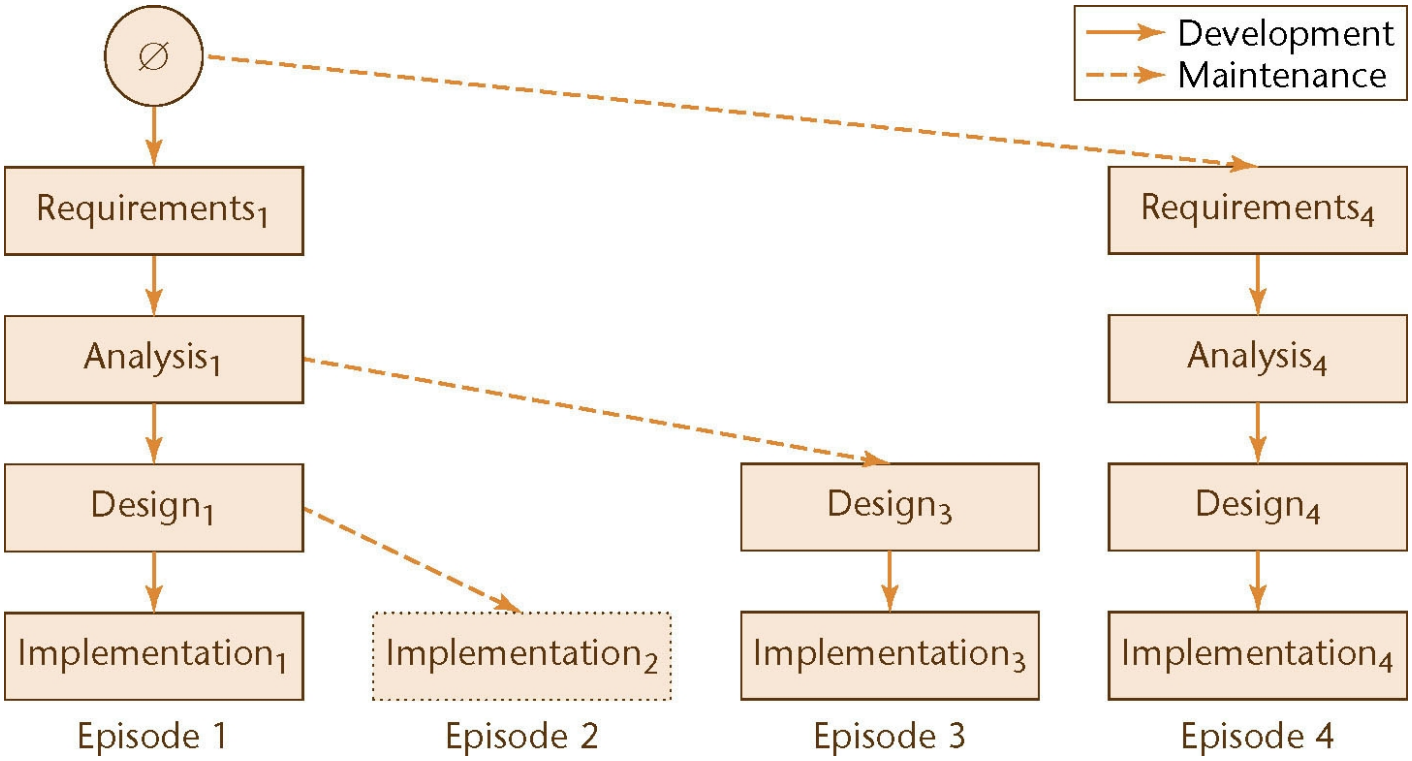
- At any one time, we can concentrate on only approximately seven *chunks* (units of information)
- To handle larger amounts of information, use *stepwise refinement*
  - Concentrate on the aspects that are currently the most important
  - Postpone aspects that are currently less critical
  - Every aspect is eventually handled, but in order of current importance
- This is an *incremental* process

Iteration and Incrementation (contd)



Iteration and Incrementation (contd)

- Iteration and incrementation are used in conjunction with one another
  - There is no single “requirements phase” or “design phase”
  - Instead, there are multiple instances of each phase



Iteration and Incrementation (contd)

- The number of increments will vary — it does not have to be four

Classical Phases versus Workflows

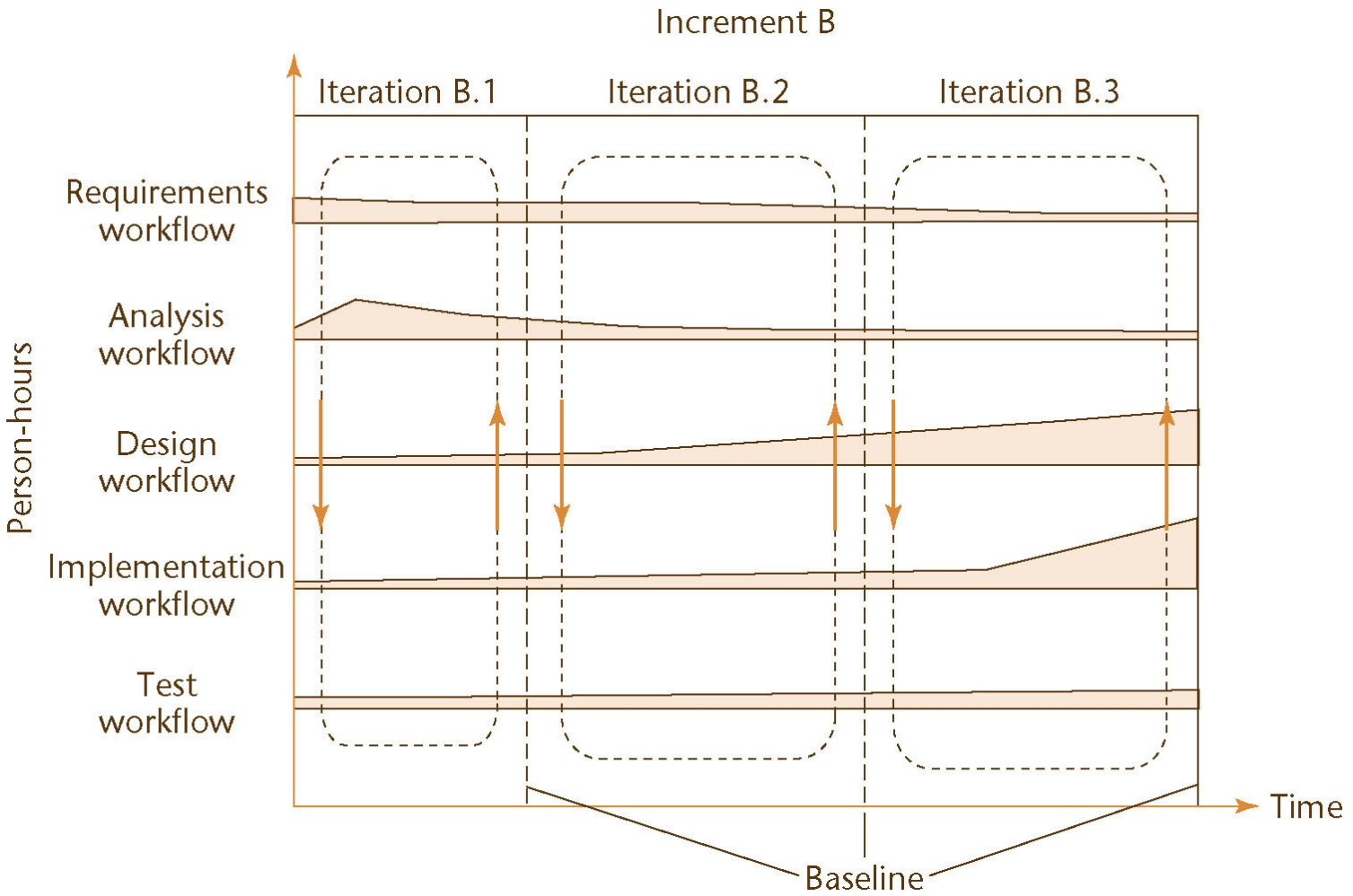
- Sequential phases do not exist in the real world
- Instead, the five core workflows (activities) are performed over the entire life cycle
  - Requirements workflow
  - Analysis workflow
  - Design workflow
  - Implementation workflow
  - Test workflow

Workflows

- All five core workflows are performed over the entire life cycle
- However, at most times one workflow predominates
- Examples:
  - At the beginning of the life cycle
    - The requirements workflow predominates
  - At the end of the life cycle
    - The implementation and test workflows predominate
- Planning and documentation activities are performed throughout the life cycle

Iteration and Incrementation (contd)

- Iteration is performed during each incrementation



Risks and Other Aspects of Iter. and Increm.

- We can consider the project as a whole as a set of mini projects (increments)
- Each mini project extends the
  - Requirements artifacts
  - Analysis artifacts
  - Design artifacts
  - Implementation artifacts
  - Testing artifacts
- The final set of artifacts is the complete product

Risks and Other Aspects of Iter. and Increm. (contd)

- During each mini project we
  - Extend the artifacts (incrementation);
  - Check the artifacts (test workflow); and
  - If necessary, change the relevant artifacts (iteration)

Risks and Other Aspects of Iter. and Increm. (contd)

- Each iteration can be viewed as a small but complete waterfall life-cycle model
- During each iteration we select a portion of the software product
- On that portion we perform the
  - Classical requirements phase
  - Classical analysis phase
  - Classical design phase
  - Classical implementation phase

Strengths of the Iterative-and-Incremental Model

- There are multiple opportunities for checking that the software product is correct
  - Every iteration incorporates the test workflow
  - Faults can be detected and corrected early
- The robustness of the architecture can be determined early in the life cycle
  - *Architecture* — the various component modules and how they fit together
  - *Robustness* — the property of being able to handle extensions and changes without falling apart

Strengths of the Iterative-and-Incremental Model (contd)

- We can *mitigate* (resolve) risks early
  - Risks are invariably involved in software development and maintenance
- We have a working version of the software product from the start
  - The client and users can experiment with this version to determine what changes are needed
- Variation: Deliver partial versions to smooth the introduction of the new product in the client organization

Strengths of the Iterative-and-Incremental Model (contd)

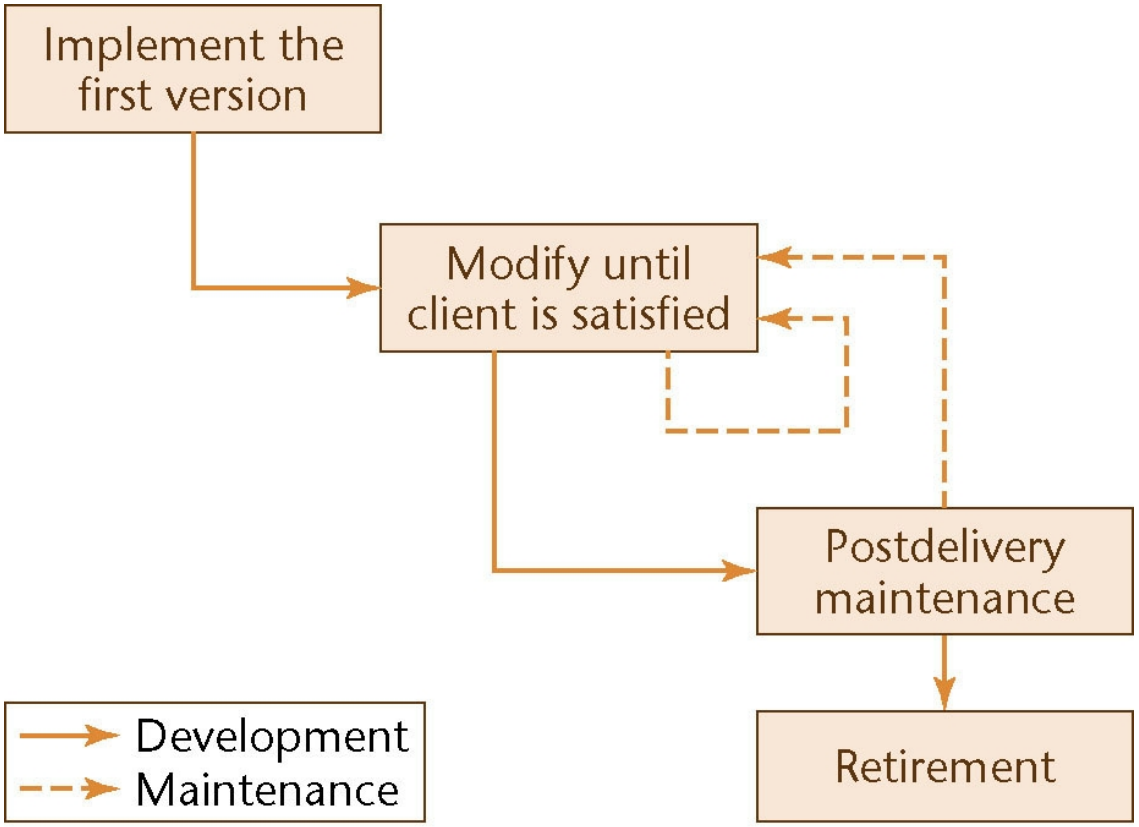
- There is empirical evidence that the life-cycle model works
- The CHAOS reports of the Standish Group (see overleaf) show that the percentage of successful products increases

Other Life-Cycle Models

- The following life-cycle models are presented and compared:
  - Code-and-fix life-cycle model
  - Waterfall life-cycle model
  - Rapid prototyping life-cycle model
  - Open-source life-cycle model
  - Agile processes
  - Synchronize-and-stabilize life-cycle model
  - Spiral life-cycle model

Code-and-Fix Model

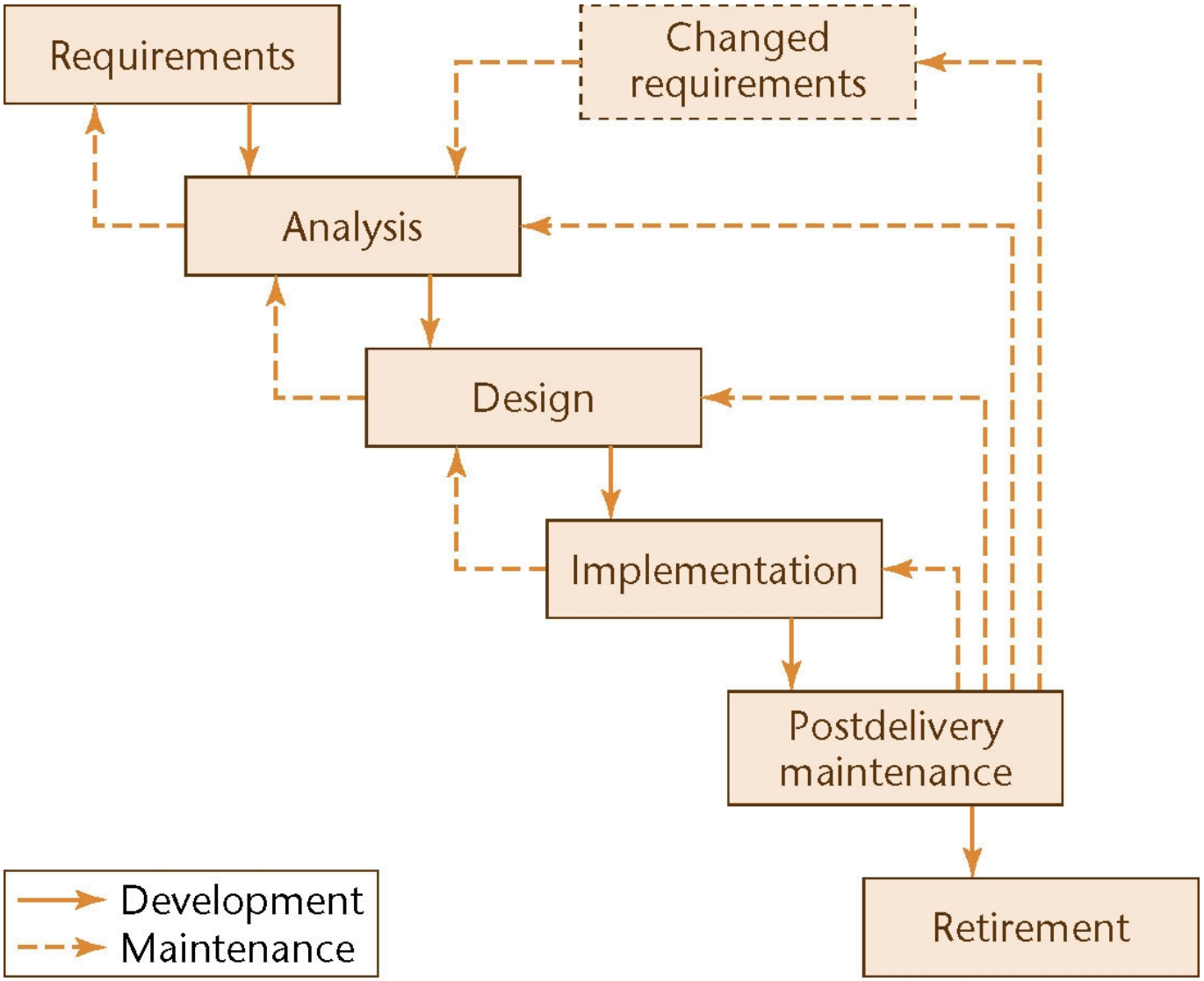
- No design
- No specifications
  - Maintenance nightmare



Code-and-Fix Model (contd)

- The easiest way to develop software
- The most expensive way

Waterfall Model

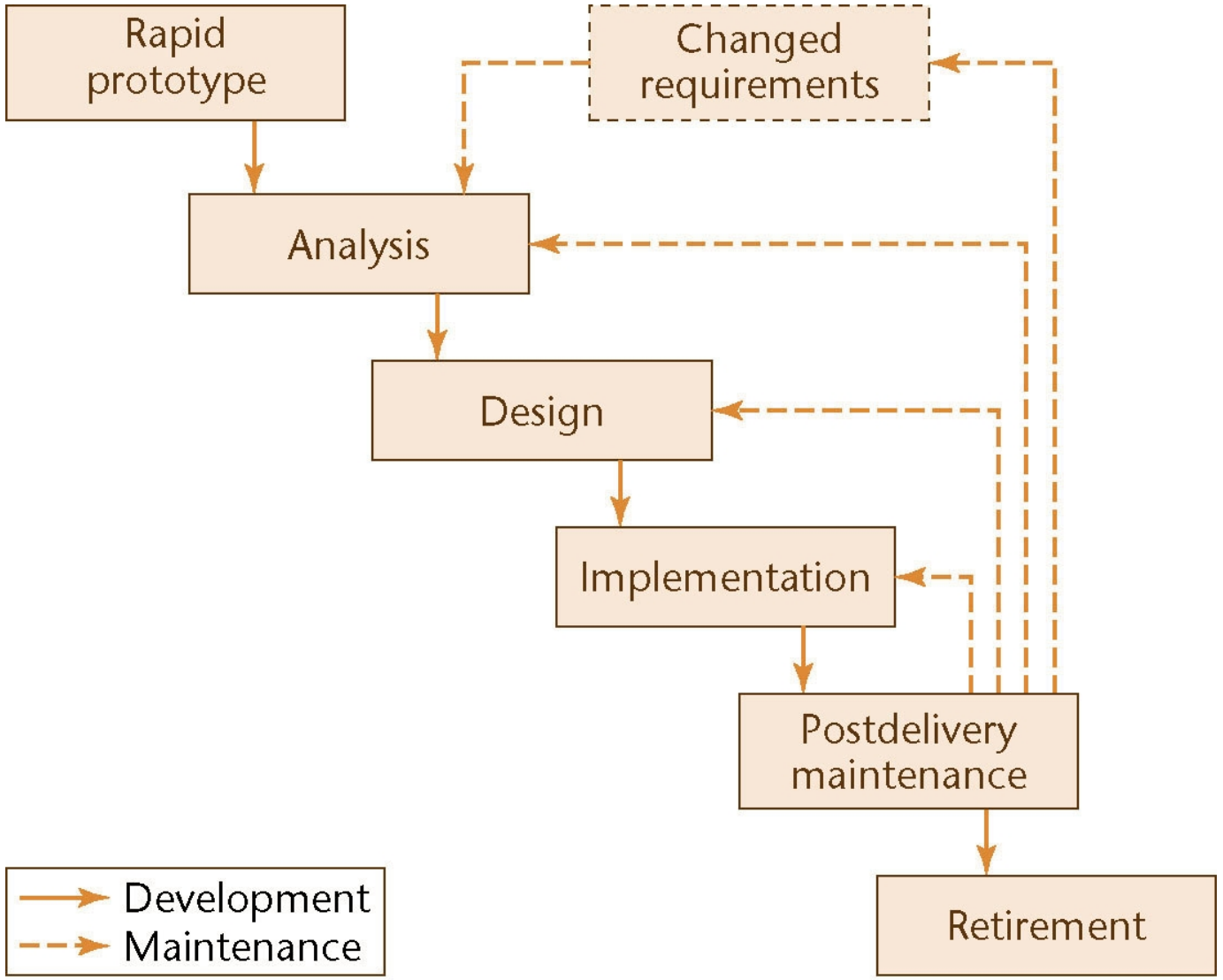


Waterfall Model (contd)

- Characterized by
  - Feedback loops
  - Documentation-driven
- Advantages
  - Documentation
  - Maintenance is easier
- Disadvantages
  - Specification document
    - Joe and Jane Johnson
    - Mark Marberry

Rapid Prototyping Model

- Linear model
- "Rapid"



Open-Source Life-Cycle Model

- Two informal phases
- First, one individual builds an initial version
  - Made available via the Internet (e.g., SourceForge.net)
- Then, if there is sufficient interest in the project
  - The initial version is widely downloaded
  - Users become co-developers
  - The product is extended
- Key point: Individuals generally work voluntarily on an open-source project in their spare time

The Activities of the Second Informal Phase

- Reporting and correcting defects
  - Corrective maintenance
- Adding additional functionality
  - Perfective maintenance
- Porting the program to a new environment
  - Adaptive maintenance
- The second informal phase consists *solely* of postdelivery maintenance
  - The word “co-developers” on the previous slide should rather be “co-maintainers”

Agile Processes

- Somewhat controversial new approach
- *Stories* (features client wants)
  - Estimate duration and cost of each story
  - Select stories for next build
  - Each build is divided into tasks
  - Test cases for a task are drawn up first
- Pair programming
- Continuous integration of tasks

Unusual Features of XP

- The computers are put in the center of a large room lined with cubicles
- A client representative is always present
- Software professionals cannot work overtime for 2 successive weeks
- No specialization
- *Refactoring* (design modification)

Acronyms of Extreme Programming

- YAGNI (you aren't gonna need it)
- DTSTTCPW (do the simplest thing that could possibly work)
- A principle of XP is to minimize the number of features
  - There is no need to build a product that does any more than what the client actually needs

Agile Processes

- XP is one of a number of new paradigms collectively referred to as *agile processes*
- Seventeen software developers (later dubbed the "Agile Alliance") met at a Utah ski resort for two days in February 2001 and produced the *Manifesto for Agile Software Development*
- The Agile Alliance did not prescribe a specific life-cycle model
  - Instead, they laid out a group of underlying principles

Agile Processes

- Agile processes are a collection of new paradigms characterized by
  - Less emphasis on analysis and design
  - Earlier implementation (working software is considered more important than documentation)
  - Responsiveness to change
  - Close collaboration with the client

Agile Processes (contd)

- A principle in the *Manifesto* is
  - Deliver working software frequently
  - Ideally every 2 or 3 weeks
- One way of achieving this is to use *timeboxing*
  - Used for many years as a time-management technique
- A specific amount of time is set aside for a task
  - Typically 3 weeks for each iteration
  - The team members then do the best job they can during that time

Agile Processes (contd)

- It gives the client confidence to know that a new version with additional functionality will arrive every 3 weeks
- The developers know that they will have 3 weeks (but no more) to deliver a new iteration
  - Without client interference of any kind
- If it is impossible to complete the entire task in the timebox, the work may be reduced ("descoped")
  - Agile processes demand fixed time, not fixed features

Agile Processes (contd)

- Another common feature of agile processes is *stand-up meetings*
  - Short meetings held at a regular time each day
  - Attendance is required
- Participants stand in a circle
  - They do not sit around a table
  - To ensure the meeting lasts no more than 15 minutes

Agile Processes (contd)

- At a stand-up meeting, each team member in turn answers five questions:
  - What have I done since yesterday's meeting?
  - What am I working on today?
  - What problems are preventing me from achieving this?
  - What have we forgotten?
  - What did I learn that I would like to share with the team?

Agile Processes (contd)

- The aim of a stand-up meeting is
  - To raise problems
  - Not solve them
- Solutions are found at follow-up meetings, preferably held directly after the stand-up meeting

Agile Processes (contd)

- Stand-up meetings and timeboxing are both
  - Successful management techniques
  - Now utilized within the context of agile processes

- Both techniques are instances of two basic principles that underlie all agile methods:
  - Communication; and
  - Satisfying the client's needs as quickly as possible

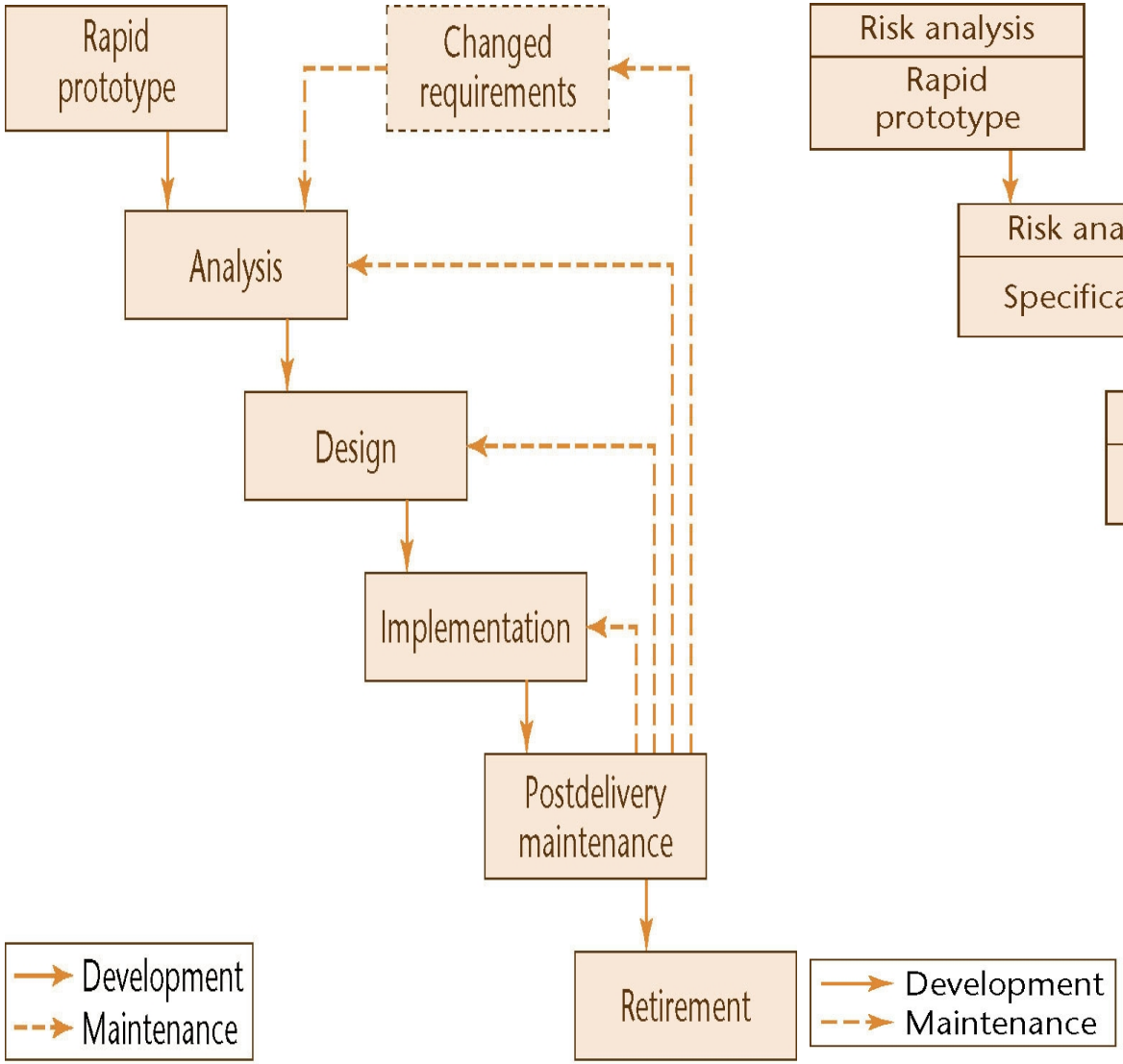
Evaluating Agile Processes

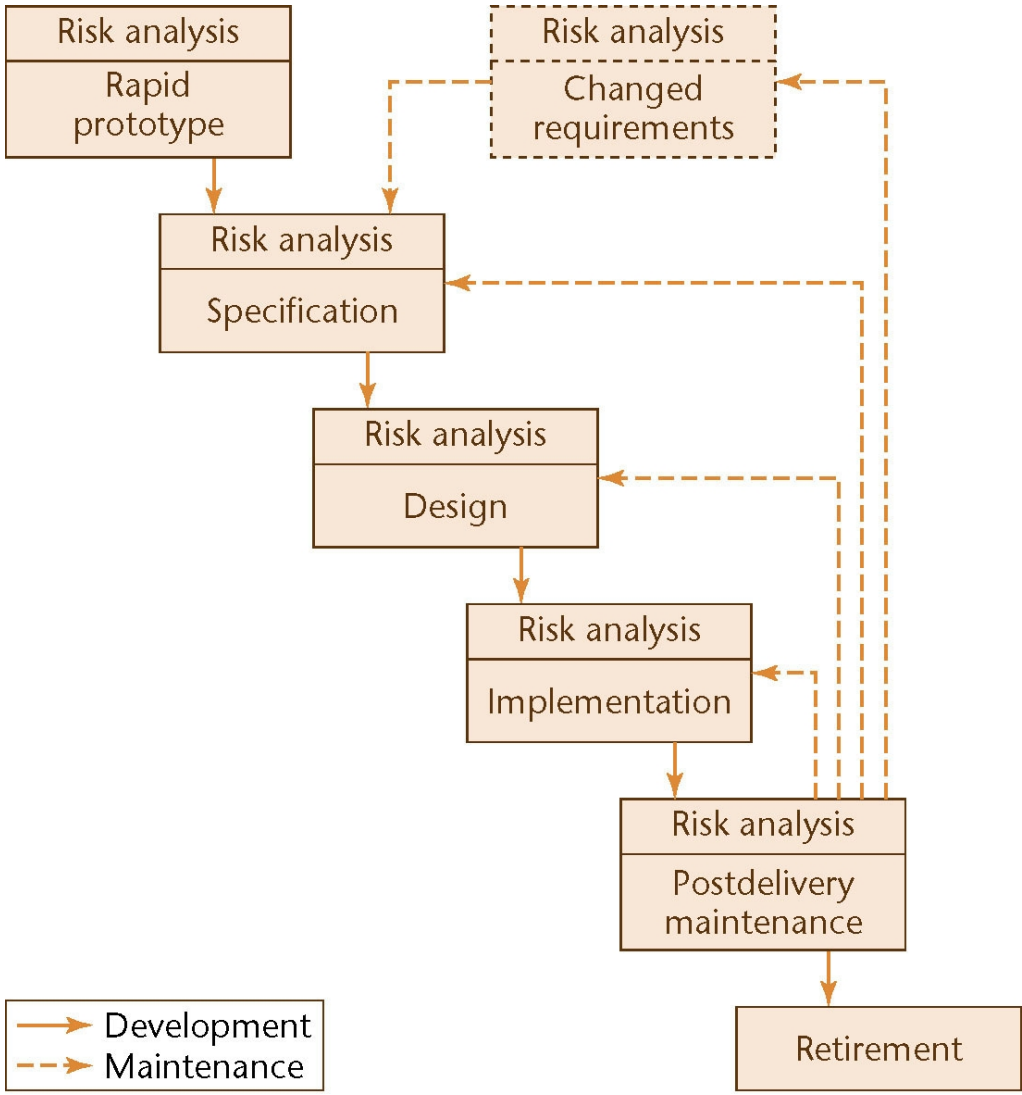
- Agile processes have had some successes with small-scale software development
  - However, medium- and large-scale software development is very different
- The key decider: the impact of agile processes on postdelivery maintenance
  - Refactoring is an essential component of agile processes
  - Refactoring continues during maintenance
  - Will refactoring increase the cost of post-delivery maintenance, as indicated by preliminary research?

Evaluating Agile Processes (contd)

- Agile processes are good when requirements are vague or changing
- It is too soon to evaluate agile processes
  - There are not enough data yet
- Even if agile processes prove to be disappointing
  - Some features (such as pair programming) may be adopted as mainstream software engineering practices

Spiral Model





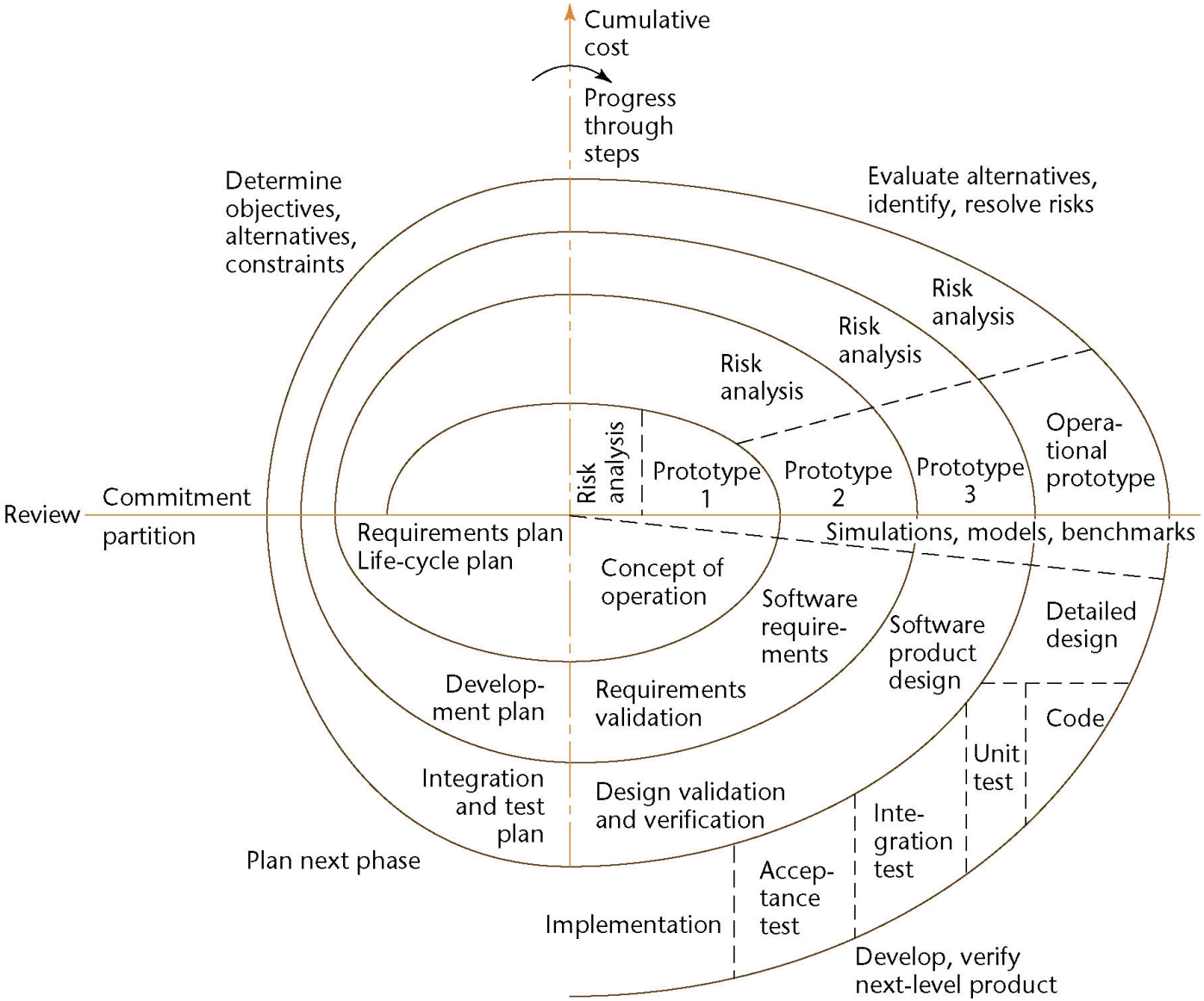
A Key Point of the Spiral Model

- If all risks cannot be mitigated, the project is immediately terminated

Full Spiral Model

- Precede each phase by
  - Alternatives
  - Risk analysis
- Follow each phase by
  - Evaluation
  - Planning of the next phase
- Radial dimension: cumulative cost to date
- Angular dimension: progress through the spiral

Full Spiral Model (contd)



Analysis of the Spiral Model

- Strengths
  - It is easy to judge how much to test
  - No distinction is made between development and maintenance
- Weaknesses
  - For large-scale software only
  - For internal (in-house) software only

Comparison of Life-Cycle Models

- Different life-cycle models have been presented
  - Each with its own strengths and weaknesses
- Criteria for deciding on a model include:
  - The organization
  - Its management
  - The skills of the employees
  - The nature of the product
- Best suggestion
  - “Mix-and-match” life-cycle model

Comparison of Life-Cycle Models (contd)

Life-Cycle Model	Strengths	Weaknesses
Evolution-tree model (Section 2.2)	Closely models real-world software production Equivalent to the iterative-and-incremental model	
Iterative-and-incremental life-cycle model (Section 2.5)	Closely models real-world software production Underlies the Unified Process	
Code-and-fix life-cycle model (Section 2.9.1)	Fine for short programs that require no maintenance	Totally unsatisfactory for nontrivial programs
Waterfall life-cycle model (Section 2.9.2)	Disciplined approach Document driven	Delivered product may not meet client's needs
Rapid-prototyping life-cycle model (Section 2.9.3)	Ensures that the delivered product meets the client's needs	Not yet proven beyond all doubt
Open-source life-cycle model (Section 2.9.4)	Has worked extremely well in a small number of instances	Limited applicability Usually does not work
Agile processes (Section 2.9.5)	Works well when the client's requirements are vague	Appears to work on only small-scale projects
Synchronize-and-stabilize life-cycle model (Section 2.9.6)	Future users' needs are met Ensures that components can be successfully integrated	Has not been widely used other than at Microsoft
Spiral life-cycle model (Section 2.9.7)	Risk driven	Can be used for only large-scale, in-house products Developers have to be competent in risk analysis and risk resolution